INFRASTRUCTURE DEVELOPER PATH · Tu amigo (Ing. Química → Linux + K8s + Vault)

Objetivo: Pasar de "Docker instalado" a "deployar sistemas seguros en K8s"

Duración total: 7 semanas de aprendizaje estructurado (10-12h/semana) **Meta**: Poder administrar Kubernetes, Vault, y monitoreo sin David

---

SEMANA 1: Linux Fundamentals (12h)

Objetivo: Dominar terminal, permisos, usuarios, procesos. Foundation para todo lo demás.

Conceptos clave

- **User/Group** = quién puede hacer qué (chmod, chown)
- **Process** = programa corriendo (pid, signal)
- **File system** = estructura de directorios (/etc, /home, /var)
- **Service/Daemon** = programa que corre en background (systemd)
- **SSH** = acceso remoto seguro (keys, not passwords)

Qué aprender (4h estudio)

1. **Linux basics** (2h)
    - "Linux in 100 seconds" (Fireship, 1.5 min)
    - https://linuxcommand.org/ — interactive shell tutorial (cap 1-10)
    - Users, permissions, file system structure
2. **Bash scripting** (1h)
    - Variables, loops, conditionals
    - Writing .sh files for automation
3. **systemd** (30min)
    - How services/daemons work
    - Starting, stopping, enabling services
4. **SSH + keys** (30min)
    - SSH key generation (ssh-keygen)
    - Config files (~/.ssh/config)
    - Key-based auth (no passwords)

Qué hacer (8h hands-on)

1. **Terminal mastery** (2h)

```bash
# User/Group basics
whoami  # Current user
id      # User ID, groups
sudo -l # What can I do as sudo?

# File permissions
ls -la  # View permissions (rwxrwxrwx)
chmod 755 script.sh  # Owner: rwx, Group: rx, Other: rx
chown user:group file  # Change ownership

# Process management
ps aux  # All processes
top     # Interactive monitoring
kill -9 <pid>  # Kill process

# Service management
systemctl list-units --type=service
systemctl start nginx
systemctl enable nginx  # Auto-start on boot
systemctl status nginx
```

2. **Bash scripting practice** (2h)

```bash
```

```bash
#!/bin/bash
# vault_startup.sh - automate Vault initialization

VAULT_ADDR="http://127.0.0.1:8200"

# Check if Vault is running
if ! curl -s $VAULT_ADDR/v1/sys/health > /dev/null; then
    echo "Starting Vault..."
    vault server -dev &
    sleep 2
fi

# Initialize secrets
vault secrets enable -path=agencia kv-v2
vault kv put agencia/test key="value"

echo "Vault ready at $VAULT_ADDR"
```

✓ Run: `bash vault_startup.sh`

3. **SSH key setup** (2h)

```bash
# Generate key
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""

# Add to WSL startup
eval $(ssh-agent -s)
ssh-add ~/.ssh/id_rsa

# Test
ssh-keyscan -H github.com >> ~/.ssh/known_hosts
ssh git@github.com  # Should work without password
```

4. **Systemd service creation** (2h)

```bash
```

```
# Create service for Vault
sudo nano /etc/systemd/system/vault.service

# Content:
[Unit]
Description=HashiCorp Vault
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/vault server -config=/etc/vault/config.hcl
Restart=always
RestartSec=5
User=vault

[Install]
WantedBy=multi-user.target

# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable vault
sudo systemctl start vault
sudo systemctl status vault
```

Checklist Semana 1

- [ ] Terminal: puedes navegar, copiar archivos, cambiar permisos sin pensar
- [ ] Bash: escribiste 1 script completo (vault_startup.sh funciona)
- [ ] SSH: key-based auth configurada, puedes SSH sin password
- [ ] systemd: creaste 1 service file, lo iniciaste/detuviste
- [ ] Documentaste: "Comandos Linux que uso cada día" (cheatsheet personal)

SEMANA 2: Docker Deep Dive (12h)

Objetivo: Construir, correr, debuggear contenedores. Dockerfile optimizados.

Conceptos clave

- **Image** = blueprint (Dockerfile + dependencies)
- **Container** = instancia corriendo de una imagen
- **Volume** = almacenamiento persistente
- **Network** = cómo hablan containers entre sí
- **Registry** = dónde viven las imágenes (Docker Hub, ECR)

Qué aprender (4h estudio)

1. **Docker fundamentals** (2h)
   - "Docker in 100 seconds" (Fireship)
   - https://docs.docker.com/get-started/ — official tutorial
   - Dockerfile, images, containers, networks
2. **Dockerfile best practices** (1h)
   - Layer caching optimization
   - Multi-stage builds
   - Minimal images (Alpine, scratch)
3. **Docker Compose** (1h)
   - How to run multiple containers
   - Services, networks, volumes

Qué hacer (8h hands-on)

1. **Container basics** (2h)

```bash
# Run existing image
docker run -it ubuntu:latest bash
# Inside container:
apt update && apt install -y curl
curl https://google.com
exit

# Run with volume
docker run -v /home/you/data:/data -it ubuntu bash
# Inside: data is mounted at /data

# List, inspect, remove
docker ps -a
docker inspect <container_id>
docker rm <container_id>
```

2. **Build custom image** (2h)

```dockerfile
```

```
# Dockerfile (for n8n with custom setup)
FROM n8nio/n8n:latest

# Install additional tools
RUN apk add --no-cache git curl jq

# Copy configuration
COPY ./n8n-config.json /home/node/.n8n/

# Expose port
EXPOSE 5678

# Health check
HEALTHCHECK --interval=30s --timeout=3s \
  CMD curl -f http://localhost:5678/api/health || exit 1
```

bash

```
# Build
docker build -t my-n8n:1.0 .

# Run
docker run -p 5678:5678 my-n8n:1.0
```

3. **Docker Compose (multi-container)** (2h)

yaml

```yaml
# docker-compose.yml
version: '3.8'

services:
  vault:
    image: vault:latest
    ports:
      - "8200:8200"
    environment:
      VAULT_DEV_ROOT_TOKEN_ID: "mytoken"
    volumes:
      - vault_data:/vault/data

  n8n:
    image: n8nio/n8n:latest
    depends_on:
      - vault
    ports:
      - "5678:5678"
    environment:
      VAULT_ADDR: "http://vault:8200"

  postgres:
    image: postgres:14
    environment:
      POSTGRES_DB: agencia
      POSTGRES_PASSWORD: devpass
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  vault_data:
  postgres_data:
```

bash

```bash
# Run all services
docker-compose up -d
# Check status
docker-compose ps
# Stop all
docker-compose down
```

4. **Debugging containers** (2h)

```bash
# View logs
docker logs -f <container_id>

# Execute command in running container
docker exec -it <container_id> bash

# Check resource usage
docker stats

# Build with debug output
docker build -t my-image:debug . --progress=plain
```

Checklist Semana 2

- [ ] Instalaste Docker + Docker Compose
- [ ] Corriste containers existentes (ubuntu, nginx, postgres)
- [ ] Escribiste 1 Dockerfile custom (funciona, no hay errores)
- [ ] Creaste docker-compose.yml con 3+ servicios
- [ ] Puedes debuggear containers (logs, exec, stats)
- [ ] Documentaste: "Mi Dockerfile best practices" (checklist personal)

SEMANA 3: Kubernetes 101 (14h)

Objetivo: Entender y deployar aplicaciones en K8s. Minikube local.

Conceptos clave

- **Cluster** = conjunto de nodos (máquinas)
- **Node** = máquina física/VM
- **Pod** = unidad mínima (1+ containers, comparten IP)
- **Deployment** = cómo deployar Pods (replicas, updates)
- **Service** = cómo exponer Pods a red
- **Namespace** = aislamiento lógico de recursos

Qué aprender (5h estudio)

1. **K8s architecture** (2h)
    - "Kubernetes in 100 seconds" (Fireship)
    - https://kubernetes.io/docs/tutorials/kubernetes-basics/ — interactive
    - Pods, Deployments, Services, Namespaces

2. **kubectl commands** (2h)
   - Getting info (get, describe, logs)
   - Creating resources (apply, create)
   - Debugging (exec, port-forward)
3. **YAML manifests** (1h)
   - How to write Pod, Deployment, Service YAML

Qué hacer (9h hands-on)

1. **Minikube setup** (1h)

```bash
# Install
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start
minikube start --cpus=4 --memory=8192 --driver=docker

# Verify
kubectl get nodes
kubectl get pods --all-namespaces
```

2. **Pod basics** (1.5h)

```yaml
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

```bash
```

```bash
kubectl apply -f pod.yaml
kubectl get pods
kubectl describe pod my-pod
kubectl logs my-pod
kubectl delete pod my-pod
```

3. **Deployment (replicas + rolling updates)** (2h)

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: app
        image: nginx:latest
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
```

```bash
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl scale deployment my-app --replicas=5
kubectl set image deployment/my-app app=nginx:1.23
```

4. **Service (expose to network)** (2h)

```yaml
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP  # Internal only
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 80
---
# LoadBalancer version (external access)
apiVersion: v1
kind: Service
metadata:
  name: my-service-lb
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 80
```

```bash
kubectl apply -f service.yaml
kubectl get services
kubectl port-forward svc/my-service 8080:80
# Now: http://localhost:8080
```

5. **Namespaces + multi-tenancy** (2.5h)

```bash
```

```
# Create namespaces
kubectl create namespace client-001
kubectl create namespace client-002

# Deploy to specific namespace
kubectl apply -f deployment.yaml -n client-001

# List resources in namespace
kubectl get all -n client-001

# Delete namespace (cascades to all resources)
kubectl delete namespace client-001
```

Checklist Semana 3

- [ ] Minikube corriendo con 4+ CPUs, 8GB RAM
- [ ] Deployed 5+ Pods (uno por imagen: nginx, postgres, vault, etc.)
- [ ] Created 1 Deployment con replicas (scale, rolling update)
- [ ] Created Services (ClusterIP, LoadBalancer)
- [ ] Created 3 namespaces, deployó en cada una
- [ ] Puedes inspeccionar recursos (get, describe, logs)
- [ ] Documentaste: "K8s cheatsheet" (kubectl commands)

SEMANA 4: Vault (Production Secrets Management) (12h)

Objetivo: Vault en K8s. Credential rotation. Kubernetes authentication.

Conceptos clave

- **Secret** = API key, password, token (NUNCA en plaintext)
- **Engine** = tipo de secrets (kv, database, pki, etc.)
- **Auth method** = cómo autenticarse con Vault (token, Kubernetes, JWT)
- **Policy** = quién puede acceder a qué
- **Dynamic secrets** = auto-rotated credentials

Qué aprender (4h estudio)

1. **Vault concepts** (2h)
    - https://www.vaultproject.io/docs — architecture
    - Auth methods, secret engines, policies
    - Kubernetes auth method

2. **Helm + Vault chart** (1h)
    - How to deploy Vault in K8s via Helm
3. **Secret rotation** (1h)
    - Dynamic secrets
    - Automated rotation strategies

Qué hacer (8h hands-on)

1. **Vault in Minikube** (3h)

```bash
# Add Helm repo
helm repo add hashicorp https://helm.releases.hashicorp.com

# Install Vault
helm install vault hashicorp/vault \
  --namespace vault --create-namespace \
  --values vault-values.yaml

# Wait for pods
kubectl get pods -n vault

# Initialize Vault
kubectl exec vault-0 -n vault -- vault operator init \
  -key-shares=1 -key-threshold=1 \
  -recovery-shares=1 -recovery-threshold=1

# Unseal
kubectl exec vault-0 -n vault -- vault operator unseal \
  <recovery_key>
```

2. **Create secrets** (2h)

```bash
```

```bash
# Enable KV v2
kubectl exec vault-0 -n vault -- vault secrets enable -path=agencia kv-v2

# Store secrets
kubectl exec vault-0 -n vault -- \
  vault kv put agencia/gemini \
  api_key="sk-..." \
  model="gemini-3.1-flash-lite-preview"

# Read secrets
kubectl exec vault-0 -n vault -- \
  vault kv get agencia/gemini
```

3. **Kubernetes auth method** (2h)

```bash
# Enable Kubernetes auth
kubectl exec vault-0 -n vault -- \
  vault auth enable kubernetes

# Configure it
kubectl exec vault-0 -n vault -- \
  vault write auth/kubernetes/config \
    kubernetes_host="https://$KUBERNETES_SERVICE_HOST:$KUBERNETES_SERVICE_PORT" \
    kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
    token_reviewer_jwt=@/var/run/secrets/kubernetes.io/serviceaccount/token

# Create role for client-001
kubectl exec vault-0 -n vault -- \
  vault write auth/kubernetes/role/client-001 \
    bound_service_account_names=default \
    bound_service_account_namespaces=client-001 \
    policies=client-001 \
    ttl=24h

# Create policy (who can read what)
kubectl exec vault-0 -n vault -- \
  vault policy write client-001 - << EOF
path "agencia/data/clients/client-001/*" {
  capabilities = ["read", "list"]
}
EOF
```

4. **Credential rotation automation** (1h)

- Documentation: "How to rotate credentials every 30 days"
- Script template for rotation

Checklist Semana 4

- [ ] Vault deployed in Minikube (helm install)
- [ ] Secrets stored in Vault (KV v2 engine)
- [ ] Kubernetes auth method configured
- [ ] Created 1 policy (read only for client-001)
- [ ] Tested: Pod in client-001 can read its secrets
- [ ] Documentaste: "Vault operations runbook" (backup, unseal, rotate)

SEMANA 5: Networking + TLS (10h)

Objetivo: K8s networking. TLS certificates. Inter-pod communication.

Conceptos clave

- **CNI** (Container Network Interface) = how pods talk to each other
- **Service DNS** = each service gets a DNS name (my-service.namespace.svc.cluster.local)
- **Ingress** = how traffic enters cluster (routes by hostname/path)
- **TLS/SSL** = encryption (certificates, keys)
- **NetworkPolicy** = firewall rules between namespaces/pods

Qué aprender (3h estudio)

1. **K8s networking** (1.5h)
    - How pods communicate (overlay network, CNI plugins)
    - Service DNS, Ingress basics
2. **TLS certificates** (1h)
    - How TLS works (public key, private key, certificate)
    - Self-signed certs vs CA-signed
    - cert-manager automation
3. **NetworkPolicy** (30min)
    - Firewall rules for K8s (deny by default, allow specific)

Qué hacer (7h hands-on)

1. **Service DNS + networking** (2h)

```bash
# Deploy 2 services
kubectl create deployment web --image=nginx -n client-001
kubectl create deployment api --image=nginx -n client-001

# Expose via Services
kubectl expose deployment web --port=80 -n client-001
kubectl expose deployment api --port=8080 --target-port=80 -n client-001

# Test inter-service communication
kubectl exec deployment/web -n client-001 -- \
  curl http://api.client-001.svc.cluster.local:8080
# Should work!
```

2. **TLS certificate** (2h)

```bash
```

```
# Generate self-signed cert (for testing)
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=agencia.local"

# Create K8s Secret
kubectl create secret tls tls-secret \
  --cert=tls.crt --key=tls.key -n client-001

# Use in Ingress
kubectl apply -f - << 'EOF'
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  namespace: client-001
spec:
  tls:
  - hosts:
    - agencia.local
    secretName: tls-secret
  rules:
  - host: agencia.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web
            port:
              number: 80
EOF
```

3. **NetworkPolicy (multi-tenant isolation)** (2h)

```yaml
```

```yaml
# networkpolicy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: client-isolation
  namespace: client-001
spec:
  podSelector: {}  # applies to all pods in namespace
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: client-001
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          name: client-001
  - to:  # Allow DNS
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - protocol: UDP
      port: 53
```

```bash
kubectl label namespace client-001 name=client-001
kubectl apply -f networkpolicy.yaml

# Test isolation
kubectl exec -it deployment/web -n client-001 -- curl http://pod-in-client-002  #
```

4. **Ingress controller** (1h)

```bash
```

```
# Enable in Minikube
minikube addons enable ingress

# Verify
kubectl get pods -n ingress-nginx
```

Checklist Semana 5

- [ ] Services en K8s con DNS funcionando (inter-service calls OK)
- [ ] Self-signed TLS certs generadas y configuradas
- [ ] Ingress controller habilitado
- [ ] NetworkPolicy configurada para aislamiento entre namespaces
- [ ] Testeaste: Pod en client-001 NO puede alcanzar client-002 (timeout)
- [ ] Documentaste: "Networking troubleshooting guide"

SEMANA 6: Monitoring + ELK (14h)

Objetivo: Ver qué está pasando. Logs centralizados. Alertas.

Conceptos clave

- **Elasticsearch** = database para logs (search, aggregate)
- **Kibana** = visualization + dashboards
- **Filebeat/Logstash** = log collectors
- **Prometheus** = metrics (CPU, memory, latency)
- **Alert** = when metric exceeds threshold, notify

Qué aprender (4h estudio)

1. **ELK Stack** (2h)
    - How logs flow: Filebeat → Logstash → Elasticsearch ← Kibana
    - Kibana dashboards, searches, alerting
2. **Prometheus + Grafana** (1h)
    - Metrics (gauge, counter, histogram)
    - Alerting rules
3. **Best practices** (1h)
    - What to log, what not to log
    - Alert threshold tuning

Qué hacer (10h hands-on)

1. **Deploy ELK via Helm** (3h)

```bash
helm repo add elastic https://helm.elastic.co
helm install elasticsearch elastic/elasticsearch -n logging --create-namespace
helm install kibana elastic/kibana -n logging
helm install logstash elastic/logstash -n logging

# Wait for pods
kubectl get pods -n logging

# Port-forward Kibana
kubectl port-forward -n logging svc/kibana 5601:5601
# Access: http://localhost:5601
```

2. **Configure Logstash** (2h)

```yaml
```

```yaml
# logstash-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: logstash-pipeline
  namespace: logging
data:
  logstash.conf: |
    input {
      kubernetes {
        namespace_watch => true
      }
    }
    filter {
      if [kubernetes][namespace_name] =~ /^client-/ {
        mutate {
          add_field => { "client_id" => "%{[kubernetes][namespace_name]}" }
        }
      }
    }
    output {
      elasticsearch {
        hosts => ["elasticsearch:9200"]
        index => "k8s-logs-%{+YYYY.MM.dd}"
      }
    }
```

3. **Kibana dashboards** (2h)
   - Create dashboard: "Client activity" (messages per client)
   - Create search: "Errors in last hour"
   - Create alert: "If error rate > 10%, notify"
4. **Prometheus + Alerting** (3h)

```bash
helm repo add prometheus-community https://prometheus-community.github.io/helm-ch
helm install prometheus prometheus-community/prometheus -n monitoring --create-na
helm install grafana prometheus-community/grafana -n monitoring

# Port-forward Grafana
kubectl port-forward -n monitoring svc/grafana 3000:80
# Access: http://localhost:3000
```

Checklist Semana 6

- [ ] ELK Stack deployed (Elasticsearch, Kibana, Logstash running)
- [ ] Logs flowing into Elasticsearch (visible en Kibana)
- [ ] Created 2+ Kibana dashboards (client activity, errors)
- [ ] Prometheus + Grafana deployed
- [ ] Created 1 alert rule (e.g., "high error rate")
- [ ] Documentaste: "How to create Kibana dashboard" (step-by-step)

SEMANA 7: Integration + Disaster Recovery (12h)

Objetivo: Todo junto. Backup. Recovery. Readiness for production.

Tarea 7a: Full system integration (6h)

1. **Deploy full stack in K8s** (3h)
    - Vault in K8s ✓
    - Postgres in K8s ✓
    - n8n in K8s ✓
    - ELK stack ✓
    - OpenClaw in K8s ✓
2. **Inter-service communication** (2h)
    - n8n → Vault (read credentials)
    - n8n → Postgres (read/write)
    - Logs flow to Elasticsearch
3. **Health checks + monitoring** (1h)
    - Liveness probes (kill if not responding)
    - Readiness probes (don't route traffic if not ready)

Tarea 7b: Backup + Recovery (4h)

1. **Backup strategy** (2h)

```bash
```

```
# Backup Kubernetes state
kubectl get all -A -o yaml > k8s-backup.yaml

# Backup Vault
vault operator raft snapshot save vault-backup.snap

# Backup Elasticsearch
curl -X PUT "elasticsearch:9200/_snapshot/backup" \
  -H 'Content-Type: application/json' \
  -d'{"type": "fs", "settings": {"location": "/mnt/backup"}}'

# Backup Postgres
kubectl exec postgres-0 -- pg_dump agencia > db-backup.sql
```

2. **Recovery test** (2h)
   - Delete a namespace, recover from backup
   - Kill Vault, recover
   - Verify data integrity

Tarea 7c: Production readiness (2h)

1. **Security checklist**
   - No API keys in YAML ✓ (using Vault)
   - RBAC configured ✓
   - Network policies active ✓
   - TLS enabled ✓
2. **SLA targets**
   - Uptime: 99.5%
   - Response time: <500ms
   - Error rate: <0.1%

Checklist Semana 7

- [ ] Full stack deployed in Minikube (all 5+ services running)
- [ ] Vault → n8n → Postgres → Elasticsearch (data flow working)
- [ ] Health checks + readiness probes configured
- [ ] Backup script tested (can restore from backup)
- [ ] Recovery test passed (delete+restore works)
- [ ] Security checklist: 100%
- [ ] Documentaste: "Production deployment guide"

REFERENCIA RÁPIDA: k8s + Vault Commands

```bash
# Kubernetes
kubectl get pods -n <namespace>
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- bash
kubectl apply -f <manifest.yaml>
kubectl delete -f <manifest.yaml>

# Vault
vault status
vault secrets enable -path=<path> kv-v2
vault kv put <path>/<secret> key=value
vault kv get <path>/<secret>
vault auth list
vault policy list

# Helm
helm repo add <name> <url>
helm install <release> <chart>
helm upgrade <release> <chart>
helm uninstall <release>

# Debugging
kubectl get events -A --sort-by='.lastTimestamp'
kubectl top nodes
kubectl top pods -n <namespace>
```

EVALUACIÓN: ¿Estoy listo?

Checklist de competencia (por semana):

- [ ] Sem 1: Terminal, bash, SSH, systemd sin pensar
- [ ] Sem 2: Puedo escribir Dockerfile + docker-compose.yml
- [ ] Sem 3: Puedo deployar aplicaciones en K8s (Deployments, Services)
- [ ] Sem 4: Vault setup, credential management, rotación básica
- [ ] Sem 5: K8s networking, TLS, NetworkPolicy
- [ ] Sem 6: Logs centralizados, dashboards, alerting
- [ ] Sem 7: Full stack integration, backups, recovery

Si todos checkean ✓ → Ready para administrar infraestructura en producción

---

Semana 1 start date: Lunes 7 de abril **Target completion**: Viernes 25 de abril **Success metric**: Puedas deployar y administrar un cluster Kubernetes con 5+ servicios sin bloqueadores.

Good luck. Esta es tu carrera ahora.